Introduction - DEVICE_PRIVATE pages

- ZONE_DEVICE pages are struct pages
- Mapped using special PTE entries (soon to be renamed softleaf)
- Mappings are created via driver initiated migration
- Used to represent device memory that is un-addressable/inaccessible from the CPU

Progress since LSFMM

- Clean-ups - page refcounts are now zero-based for all ZONE_DEVICE pages
- Support for other large ZONE_DEVICE folios has been merged
- https://lore.kernel.org/linux-mm/20251001065707.920170-1-balbirs@nvidia.com/

Problems with the current implementation of DEVICE_PRIVATE pages

- The kernel can't really just steal physical address space - causes issues with KASLR amoung other things
- Some platforms don't make room in the linear map (eg. AArch64)
- Initialisation of struct page metadata takes a long time - 10s of seconds with larger amounts of device memory
- And of course memory usage overhead
- P2PDMA pages can double this usage and initialisation time

Why do we need a struct page?

page_to_pfn() and pfn_to_page()

- Lots of code uses page_to_pfn() and pfn_to_page()
- Is it really needed? Device private pfns are only obtained from special swap entries or device private pages
- So instead we could look up associated pages/folios using special code paths
- Implemented in [RFC PATCH 0/6] Remove device private pages from physical address space
  - https://lore.kernel.org/linux-mm/20251128044146.80050-1-jniethe@nvidia.com/
- Solves the issues on AArch64 and other platforms

Drawbacks of this approach

- Some code (eg. page table walkers) that use PFNs may see aliased "PFNs"
- Should we mark these somehow? We could for example set the top bit of device "PFNs"/offsets
- Could add a type for device PFNs/offsets, but only two places have been identified so that seems like overkill
- Still relies on struct page, so has all the initialisation and memory overhead associated with that

```
struct page_vma_mapped_walk {
        unsigned long pfn;
        unsigned long nr_pages;
        pgoff_t pgoff;
        struct vm_area_struct *vma;
        unsigned long address;
        pmd_t *pmd;
        pte_t *pte;
        spinlock_t *ptl;
        unsigned int flags;
};
```

What else do DEVICE_PRIVATE pages use struct page for?

- Page flags
  - Pages are mapped via migration - so migrated pages need to hold relevant state
  - What flags are actually needed? This is anonymous memory so probably only PG_dirty, PG_locked and PG_anon_exclusive
- Mapping information - ie. page->mapping, index and private fields
- Driver specific data pointer - page->zone_device_data
- Reference/mapped counts
- RMAP lookup
- Migration entry lock

Can use a device-specific struct to hold this information?

- Do we really need to support multiple mappings of the same DEVICE_PRIVATE page? Or can these always be anon exclusive?
- Does this also apply to P2PDMA pages - there is often redundancy here because many devices also expose memory via PCIe BARs
- Maybe the folio structs could be some smaller data structure covering just what device drivers and core-mm need?
- What should the device-specific struct look like? Is there any overlap with the future move towards memory descriptors?
- Or do we just wait for struct page removal and re-use dynamically allocated struct folios?

Matthew's proposed struct folio

https://kernelnewbies.org/MatthewWilcox/Memdescs

```
struct folio {
    unsigned long flags;
    struct list_head lru;
    struct address_space *mapping;
    pgoff_t index;
    void *private;
    atomic_t _refcount;
    atomic_t _mapcount;
    atomic_t pincount;
    unsigned char order;
    unsigned long pfn;
    unsigned long memcg_data;
};
```

Strawman struct device_folio

```
struct device_folio {
  unsigned long flags;
  unsigned char order;
  unsigned long device_offset;
};
```